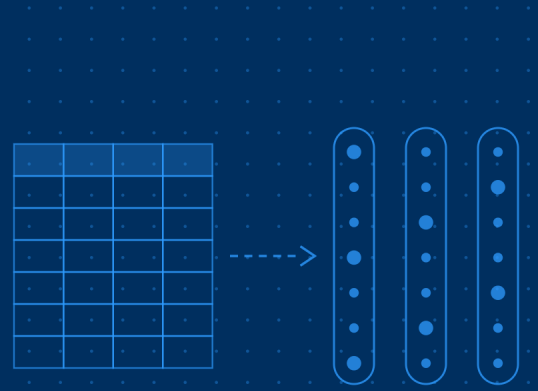
UE INGÉNIERIE DES DONNÉES · MODULE 14

TD · 3 h · 7 notebooks · 4 approches · 5 modèles

Feature engineering : le TD

Quatre façons de préparer les mêmes réservations, cinq modèles pour les juger.



Python · pandas · scikit-learn · Jupyter · DuckDB · Feast · GitHub



Abraham Zacharie Koloboe
Cloud Data Engineer

TD : mercredi 21 octobre 2026

OBJECTIFS

Deux cours, une question, six savoir-faire

00

Cadrer

repérer les fuites, découper dans le temps

`decouper()`

01 · A

Encoder

selon l'ordre, la cardinalité et le modèle

`OneHotEncoder` `TargetEncoder`

02 · B

Créer

une variable = une hypothèse vérifiée

`merge_asof`

03 · C

Réduire, sélectionner

ACP, filtre, wrapper, L1

`PCA` `SelectFromModel`

04 · D

Adapter au modèle

ce qu'un arbre n'exige pas

`HistGradientBoosting`

05 06

Industrialiser

un pipeline, un feature store

`Pipeline` `Feast`

Le fil rouge : seule la préparation change, on mesure ce qu'elle rapporte.

LE CAS

Prédire l’annulation dès la réservation

119 208
réservations réelles

2
hôtels portugais · Lisbonne et Algarve

26
mois d’arrivées · juillet 2015 → août 2017

37,1 %
annulées (absences comprises)

Une réservation, une ligne

réservation n° 71658

City Hotel · 2 adultes, 4 nuits · réservée 21 jours avant · agent 9 · en ligne · client français · sans acompte

→ probabilité d’annulation : **0,474**

réellement annulée

Connue à la réservation ?

séjour, délai, prix, canal, client	Pas de fuite
`days_in_waiting_list`, `total_of_special_requests`	à discuter
`reservation_status` chambre attribuée, parking, modifications	Fuite

Avant chaque variable, une question : était-elle connue à l’instant de la réservation ?

VUE D'ENSEMBLE

Un cadre, quatre approches, un verdict

LE CADRE

00 Cadrer

- la cible `is_canceled`
- 5 fuites retirées
- train jusqu'au 21 avril 2017, test ensuite

QUATRE PRÉPARATIONS

01 · A Le strict minimum

colonnes brutes, one-hot, standardisation

365 colonnes

02 · B Le métier d'abord

une variable = une hypothèse

83 colonnes

03 · C Tout générer, puis réduire

interactions, puis ACP, filtre ou L1

470 → 40 à 182

04 · D Laisser faire les arbres

codes entiers, aucune échelle

25 colonnes

mêmes 5 modèles :
régression logistique
kNN
réseau de neurones
forêt aléatoire
gradient boosting

LE VERDICT

05 Comparer

6 préparations × 5 modèles · un modèle retenu

POUR ALLER PLUS LOIN

06 Feature store

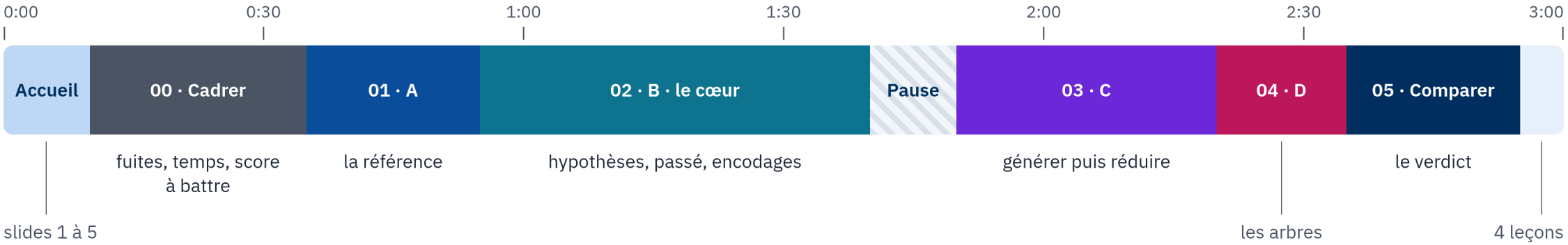
Feast : servir les mêmes variables à l'entraînement et en production

Mêmes données, même découpage, mêmes modèles : seule la préparation change.

DÉROULÉ DE LA SÉANCE

3 heures, 6 notebooks commentés, 1 en option

APRÈS LA SÉANCE notebook 06 · Feast (25 min)



on lit et on commente ensemble · question à discuter en salle · à relancer chez soi

Les notebooks sont déjà exécutés : on lit, on commente, on discute.

NOTEBOOK 00 · CADRER

Les fuites, le temps, puis le score à battre

Le crible des fuites

`reservation_status` — Fuite
AUC de **1,000**

chambre attribuée ≠ réservée — Fuite
5,4 % d'annulations contre 41,6 %

parking demandé — Fuite
0 % sur 7 409 réservations

« Non Refund » — Pas de fuite
99,4 % d'annulations

⚠ 3 fuites discrètes : 0,873 → 0,893, personne ne s'en méfie

Découper dans le temps

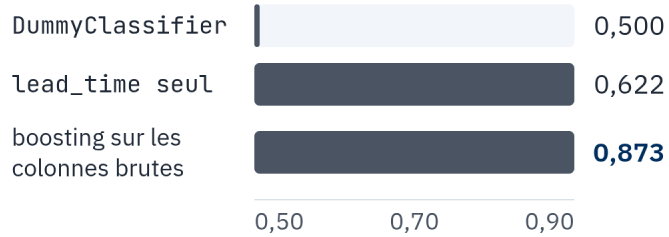
train · 95 190 · 36,1 % annulées	test 24 018 41,0 %

1er juill. 2015 → 21 avr. 2017 — 22 avr. → 31 août 2017

au hasard :
0,935
⚠ des jumelles au train

dans le temps :
0,873
OK le score à battre

Le score à battre



AUC au test, découpage dans le temps

Un score parfait est un signal d'alarme ; un découpage au hasard flatte le score.

NOTEBOOK 01 · APPROCHE A

Le brut, bien encodé : la référence à battre

`ColumnTransformer`

14 numériques → `SimpleImputer(median)` → `StandardScaler` → **14**

11 catégorielles → `OneHotEncoder(min_frequency=20)` → **351**

→ **365 colonnes**

`agent · country · company` des étiquettes, pas des nombres

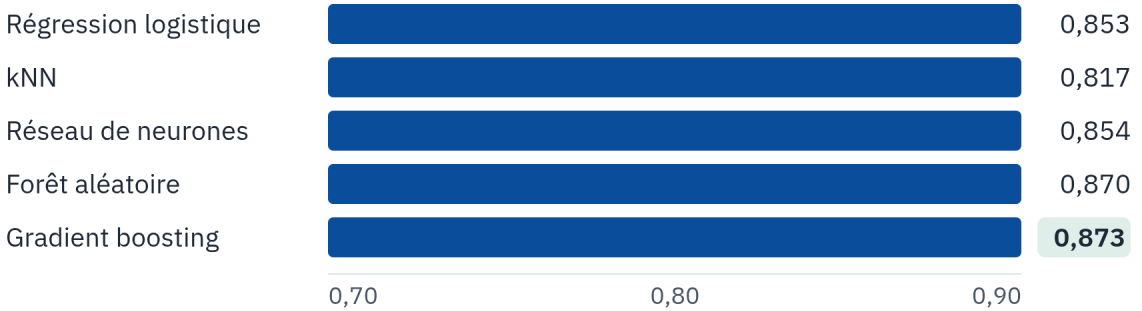
Le piège de `pd.get_dummies`

train : **165** colonnes ≠ test : **136** colonnes

OneHotEncoder : 72 colonnes au train comme au test

12 pays jamais vus → colonne « autres »

Les scores de A



fit sur le train, transform partout : l'encodeur mémorise les modalités du train.

NOTEBOOK 02 · APPROCHE B · CRÉER

Une variable, une hypothèse, une vérification

Variable	Hypothèse	Au train	Verdict
`tranche_delai`	on annule d'autant plus qu'on a réservé tôt	de 9,4 % (< 1 sem.) à 60,1 % (> 6 mois)	✓ tient nettement
`jour_resa_sin, _cos`	le jour de la réservation compte	vendredi 48,9 %, autres jours de 29,9 à 35,4 %	✓ tient
`agent_absent`	sans intermédiaire, on annule moins	21,6 % contre 38,5 %	✓ tient
`domestique` (jointure)	un client du pays annule plus facilement	Portugal 55,5 %, autres 21,2 %	✓ tient
`prix_total`	plus c'est cher, plus on s'engage	24,4 %, puis 44,1 %, puis 35,6 %	⚠ pas monotone

44
variables construites

`taux_annul_client`
99,6 % sur 5 832 lignes…
3 au test
⚠ trop beau : des blocs de groupes

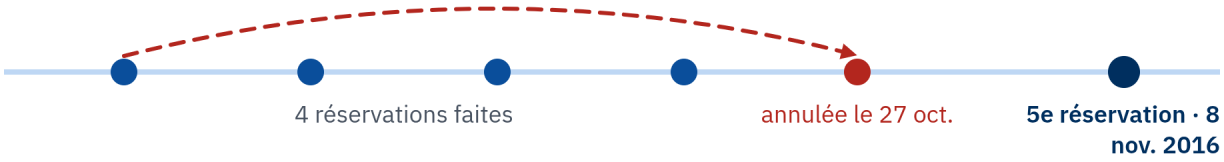
Une variable dérivée est une hypothèse qu'on énonce, puis qu'on vérifie sur le train.

NOTEBOOK 02 · APPROCHE B · ENCODER

Le passé, rien que le passé, encodé par groupe

Ce qu'on savait ce jour-là : l'agent 262

à cette date : **4** faites, **1** seule close → taux à date = **1,0**




```
pd.merge_asof(gauche, closes, left_on="date_reservation",
              right_on="date_cloture", by="agent",
              allow_exact_matches=False)
```

Le piège : toute la période

petits agents, 50 réservations au plus

sur toute la période :

0,931

Fuite

à date :

0,539

Pas de fuite

Une préparation par groupe

`asym · 11 · Yeo-Johnson` `num · 21 · standardisation` `cat · 8 → 44 one-hot` `forte_card · 3 · TargetEncoder` `tranche · 1 · ordinal`

83 colonnes

Une agrégation de la cible ne compte que les issues connues à la date de la prédiction.

NOTEBOOK 03 · APPROCHE C

Tout générer, puis réduire : ça ne paie pas

Générer

14 numériques → `PolynomialFeatures(degree=2)` → **119** + 351 one-hot →

⚠ croiser aussi les one-hot : 67 160 colonnes, 51 Go

470 colonnes

Réduire

C1 · Extraire

`PCA(n_components=0.95)`

95 axes

meilleure AUC **0,819** (réseau)

C2 · Filtrer

`SelectKBest(f_classif, k=40)`

40 colonnes

meilleure AUC **0,826** (boosting)

C3 · Embedded

`SelectFromModel` + L1

182 colonnes

meilleure AUC **0,867** (boosting)

référence A : 0,873

⚠ Sélectionner avant de valider, sur du pur bruit : 1 % d'erreur annoncée, 51 % en réalité

Toute réduction apprend : elle va dans le pipeline, et elle ne remplace pas une hypothèse.

NOTEBOOK 04 · APPROCHE D

Les arbres se contentent de codes entiers

Numéroter, c'est inventer un ordre

`OrdinalEncoder` · `arrival_date_month`

```
April → 0
August → 1
December → 2
…
September → 11
```

⚠ ordre alphabétique : janvier entre février et juillet

L'échelle ne compte pas pour un arbre

forêt, sans ou avec StandardScaler :

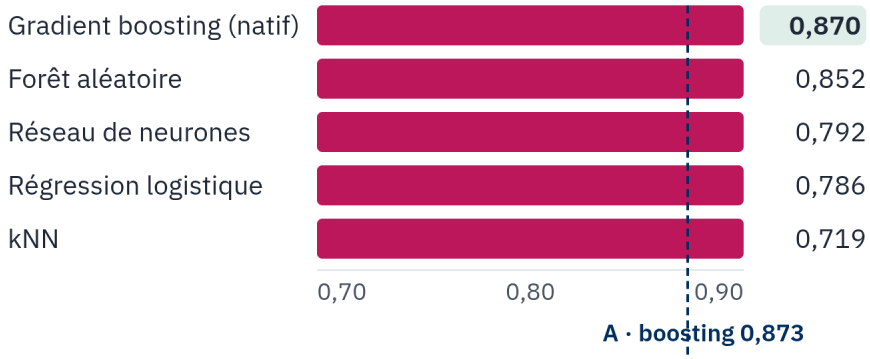
0,8524 = 0,8524 ✓

régression logistique sur échelles brutes :

2 000 itérations sur 2 000

⚠ sans converger

Les scores de D



25 colonnes · environ 3 s pour le boosting

La préparation se choisit avec le modèle : ce qui suffit aux arbres fait chuter les autres.

NOTEBOOK 05 · LE VERDICT

6 préparations × 5 modèles : le verdict

Approche	Colonnes	Rég. log.	kNN	Réseau	Forêt	Boosting
A strict minimum	365	**0,853**	0,817	0,854	0,870	0,873
B variables métier	83	0,851	**0,853**	**0,881**	**0,888**	**0,888**
C1 interactions + ACP	95	0,795	0,743	0,819	0,789	0,807
C2 interactions + filtre	40	0,808	0,803	0,815	0,819	0,826
C3 interactions + L1	182	0,845	0,762	0,850	0,837	0,867
D arbres natifs	25	0,786	0,719	0,792	0,852	0,870

Retenu : B + gradient boosting · AUC 0,888

AUC au test · du blanc (0,72) au bleu (0,89) · meilleur score de chaque colonne en vert

Le métier paie : B gagne pour 4 modèles sur 5

Générer ne suffit pas : aucun C ne bat A

Tout dépend du modèle : D égale A pour le boosting, pas pour le kNN

La meilleure préparation est celle qui sert le modèle, avec le moins de colonnes.

NOTEBOOK 06 · OPTIONNEL · FEAST

Les mêmes valeurs, à l'entraînement et en prod

1 · CALCULER

DuckDB, SQL, fonctions de fenêtre

```
SUM(...) OVER (PARTITION BY agent ...)
```

2 · PHOTOGRAPHIER

22 351 photos d'agents · 22 683 photos de pays

datées J + 1

3 · ENREGISTRER

`feast apply` → le registre

2 entités · 2 vues · 1 service

`modele_annulation_v1`

4 · SERVIR

deux sorties, ci-dessous

Entraînement

`get_historical_features`

la jointure à date, faite par l'outil

Production

`materialize` → SQLite → `get_online_features`

agent 9, au 22 avril 2017 : **29 491** réservations passées, taux **0,452**

OK écart maximal avec **merge_asof** : 0,0

⚠ 307 couples perdus sans erreur : le premier jour d'un agent

Un feature store sert des variables datées : pas de décalage entre entraînement et production.

À VOUS

Installez, ouvrez le 00, discutons

Installer

1. **Cloner**

```
git clone …/ama-feature-engineering-tp
```

2. **Installer**

```
uv sync
```

3. **Ouvrir**

```
uv run jupyter lab
```

pandas 2.3 (exigé par Feast)

```
sans uv : pip install -r requirements.txt
```

6 réflexes

- Connue à l'instant de la réservation ?
- Découper dans le temps
- fit sur le train, transform partout
- Le passé, rien que le passé
- Encoder selon l'ordre, la cardinalité, le modèle
- Tout dans un Pipeline

Retrouver

le site : notebooks exécutés, lisibles sans rien installer

https://abrahamkoloboe27.github.io/ama-feature-engineering-tp/

code

github.com/abrahamkoloboe27/ama-feature-engineering-tp

`notebooks/` `src/fe_tp/` `feature_store/` `docs/`

Prochaine étape : le notebook 00, et la première question à la salle.

La fiche réflexe du cours 2, vérifiée

Famille de modèles	Mise à l'échelle	Catégorielles	Ce qu'on a mesuré au TD
Linéaires et logistiques	nécessaire	one-hot ou encodage par la cible	codes ordinaux bruts : 0,853 → 0,786
kNN, SVM, ACP	indispensable	one-hot	de 0,719 (D) à 0,853 (B)
Réseaux de neurones	indispensable	one-hot ou plongements	meilleur avec B : 0,881
Arbres, forêts, boosting	inutile	ordinal ou support natif	boosting natif : 0,870 avec 25 colonnes

Colonnes 1 à 3 : slide 28 du cours 2 · colonne sarcelle : notebook 05, AUC au test.

Les 5 blocages les plus fréquents

Symptôme	Cause	Solution
`ModuleNotFoundError: fe_tp`	paquet du TD non installé	`uv sync`, ou `pip install -e .`
pandas 3 installé, Feast refuse	`Feast exige pandas<3`	`uv sync` (pandas 2.3)
`FileNotFoundError: modele_B.joblib`	dépôt incomplet	**git pull**, ou exécuter le 02 avant le 05
Une cellule d'évaluation tourne longtemps	5 modèles sur 95 190 réservations	attendre 1 à 4 min, ou lire la sortie déjà là
Feast : registre incohérent	`un ancien feature_store/data/`	relancer le 06 depuis le début : il le recrée



Merci, et à vos notebooks

Abraham Zacharie Koloboe · Cloud Data Engineer — questions, retours, idées : écrivez-moi.

abraham.koloboe@gmail.com

LinkedIn · Abraham Zacharie Koloboe

github.com/abrahamkoloboe27

LA SUITE

Le site du TD : les 7 notebooks exécutés, lisibles sans rien installer — abrahamkoloboe27.github.io/ama-feature-engineering-tp

Chez vous : cloner le dépôt, uv sync, uv run jupyter lab — tout réexécuter prend une quinzaine de minutes.

AMA sur LinkedIn · linkedin.com/company/bourseama